

NAME : P SAI VENKAT

ROLL NO : 2403A510G0

SUBJECT : AI ASSISTED CODING

TEST : LAB TEST -2

Lab Set 5

Question 1: AI-Assisted Code Review

Task 1:

Submit a sample Python script (with inefficient logic and poor naming) to an AI model for review.

Task 2:

Ask the AI to rewrite the code following PEP 8 guidelines and include explanations of improvements in naming, indentation, and structure

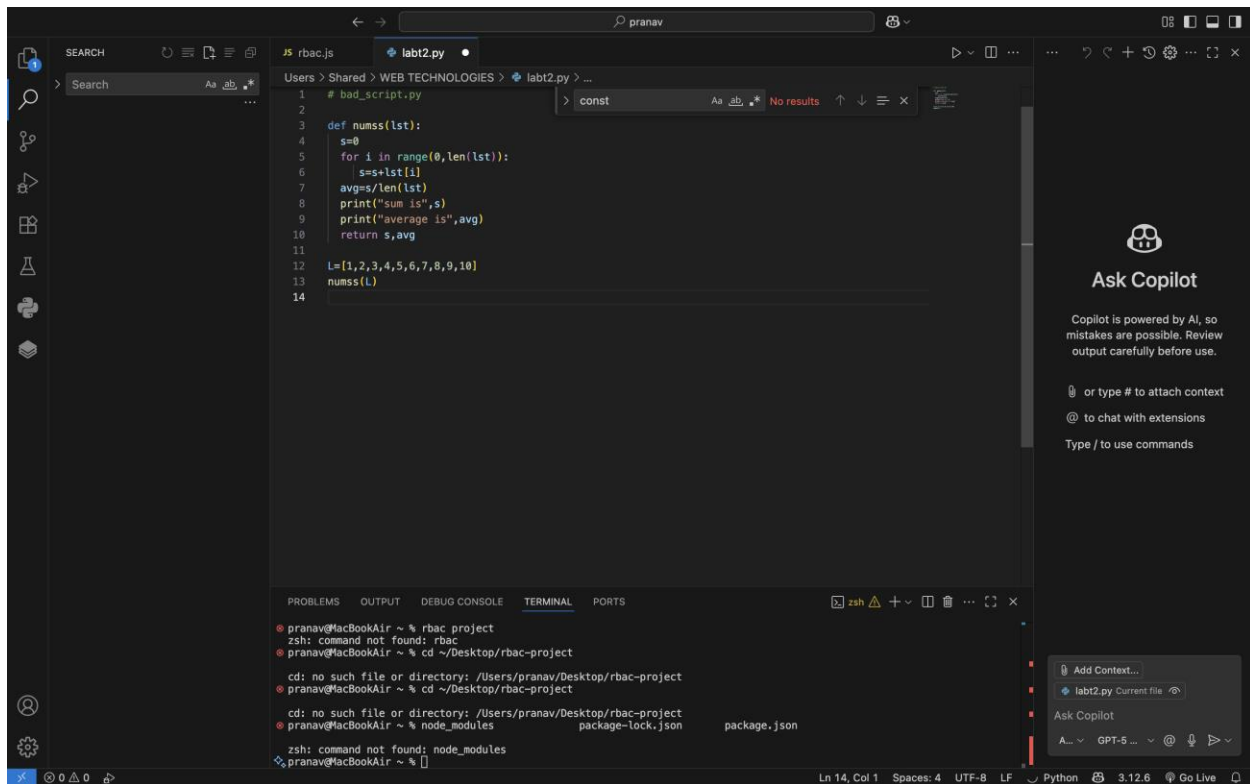
PROMPT :

Rewrite the below Python code following **PEP 8 guidelines**.

Improve **naming, indentation, and structure** for clarity and efficiency.

Explain the **changes made and why they improve the code**.

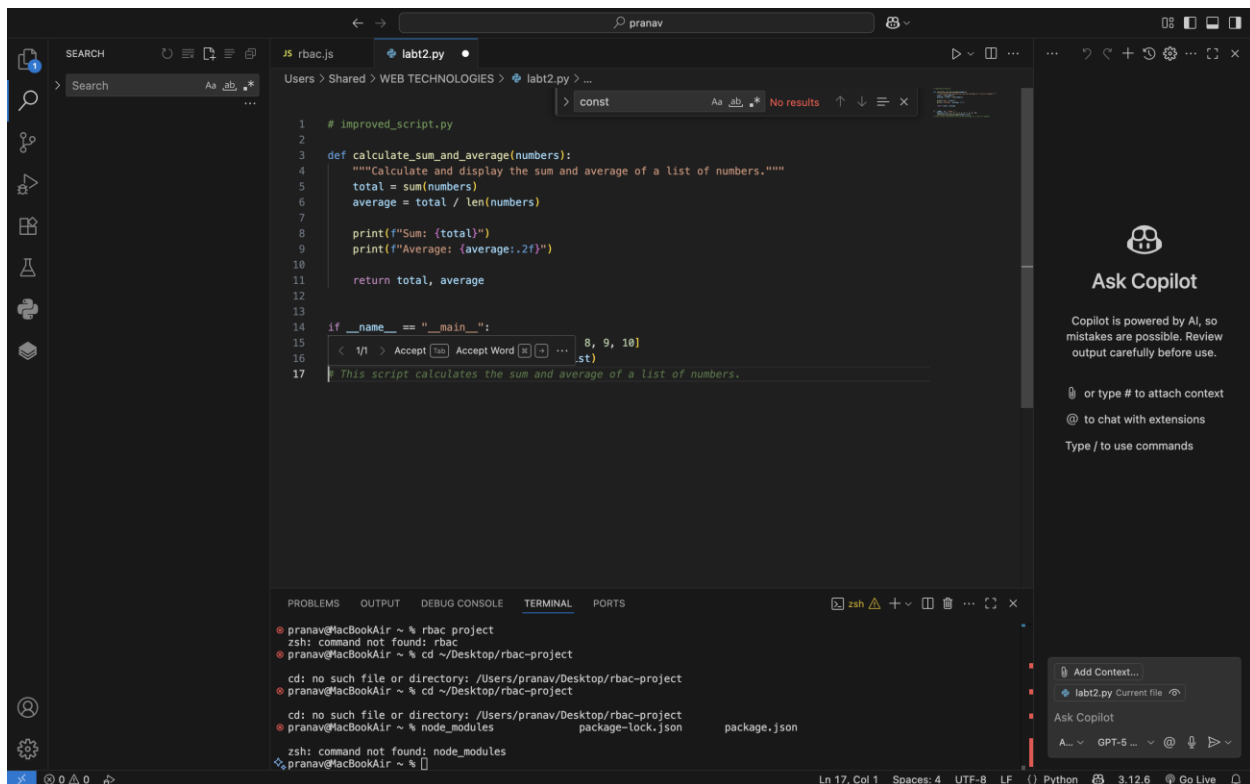
BAD PYTHON SCRIPT :



```
1 # bad_script.py
2
3 def numss(lst):
4     s=0
5     for i in range(0,len(lst)):
6         s=s+lst[i]
7     avg=s/len(lst)
8     print("sum is",s)
9     print("average is",avg)
10    return s,avg
11
12 L=[1,2,3,4,5,6,7,8,9,10]
13 numss(L)
14
```

pranav@MacBookAir ~ % rbac project
zsh: command not found: rbac
pranav@MacBookAir ~ % cd ~/Desktop/rbac-project
cd: no such file or directory: /Users/pranav/Desktop/rbac-project
pranav@MacBookAir ~ % cd ~/Desktop/rbac-project
cd: no such file or directory: /Users/pranav/Desktop/rbac-project
pranav@MacBookAir ~ % node_modules
package-lock.json
zsh: command not found: node_modules
pranav@MacBookAir ~ %

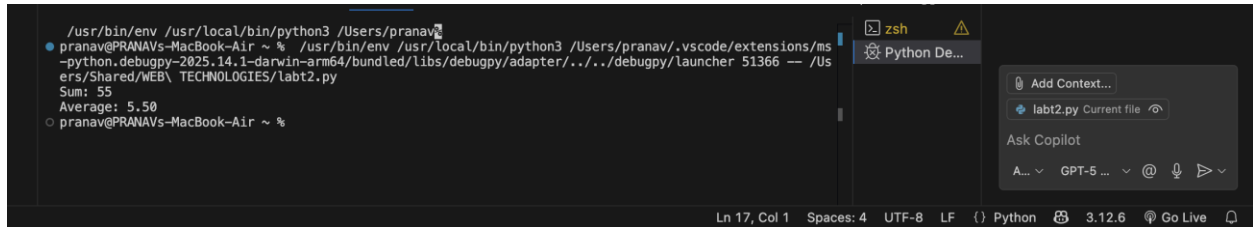
IMPROVED PYTHON SCRIPT OR CODE :



```
1 # improved_script.py
2
3 def calculate_sum_and_average(numbers):
4     """Calculate and display the sum and average of a list of numbers."""
5     total = sum(numbers)
6     average = total / len(numbers)
7
8     print(f"Sum: {total}")
9     print(f"Average: {average:.2f}")
10
11    return total, average
12
13
14 if __name__ == "__main__":
15     < 1/1 > Accept Tab Accept Word ... 8, 9, 10
16     .st)
17     This script calculates the sum and average of a list of numbers.
```

pranav@MacBookAir ~ % rbac project
zsh: command not found: rbac
pranav@MacBookAir ~ % cd ~/Desktop/rbac-project
cd: no such file or directory: /Users/pranav/Desktop/rbac-project
pranav@MacBookAir ~ % cd ~/Desktop/rbac-project
cd: no such file or directory: /Users/pranav/Desktop/rbac-project
pranav@MacBookAir ~ % node_modules
package-lock.json
zsh: command not found: node_modules
pranav@MacBookAir ~ %

OUTPUT :



```
/usr/bin/env /usr/local/bin/python3 /Users/pranav/
pranav@PRANAVs-MacBook-Air ~ % /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/ms
-python.debugpy-2025.14.1-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51366 -- /Us
ers/Shared/WEB\ TECHNOLOGIES/labt2.py
Sum: 55
Average: 5.50
pranav@PRANAVs-MacBook-Air ~ %
```

Function Name.

BEFORE :numss,AFTER : calculate_sum_and_average,

>Clear, descriptive name following snake_case convention.

Variable Names

BEFORE :s, L, i,AFTER : total, numbers_list, numbers,

>Improved readability and meaning.

Loop Logic,

BEFORE : Manual summation with for loop, AFTER :Used built-in sum() function,

> More efficient and Pythonic.

Indentation,

BEFORE : Inconsistent spacing,AFTER: 4 spaces per indent,

>Complies with PEP 8 standard.

Structure,

BEFORE : No entry point, AFTER :Added if __name__ == "__main__":, >Makes code modular and reusable.

Docstring,

BEFORE : None, AFTER :Added descriptive docstring,

> Explains function purpose clearly.

Question 2: Improving Readability and Maintainability

Task 1:

Provide AI with a long, nested function (e.g., multiple if-else or loops). Ask it to refactor the code into modular, readable functions.

Task 2:

Use AI to generate code comments and a short design summary, explaining how refactoring improved maintainability and clarity.

PROMPT :

Refactor the following long, nested Python function into **modular and readable functions**.

Make the code easier to understand and maintain.

Add **clear comments** and provide a **short design summary** explaining how refactoring improved readability and maintainability.

INITIAL CODE :

```
1 def process_data(data):
2     total = 0
3     count = 0
4     for item in data:
5         if item > 0:
6             if item % 2 == 0:
7                 total += item
8             else:
9                 total += item * 2
10            count += 1
11        else:
12            if item == 0:
13                print("Zero found")
14            else:
15                print("Negative number ignored")
16    avg = total / count if count > 0 else 0
17    print("Total:", total)
18    print("Average:", avg)
19
20
```

Sum: 55
Average: 5.50

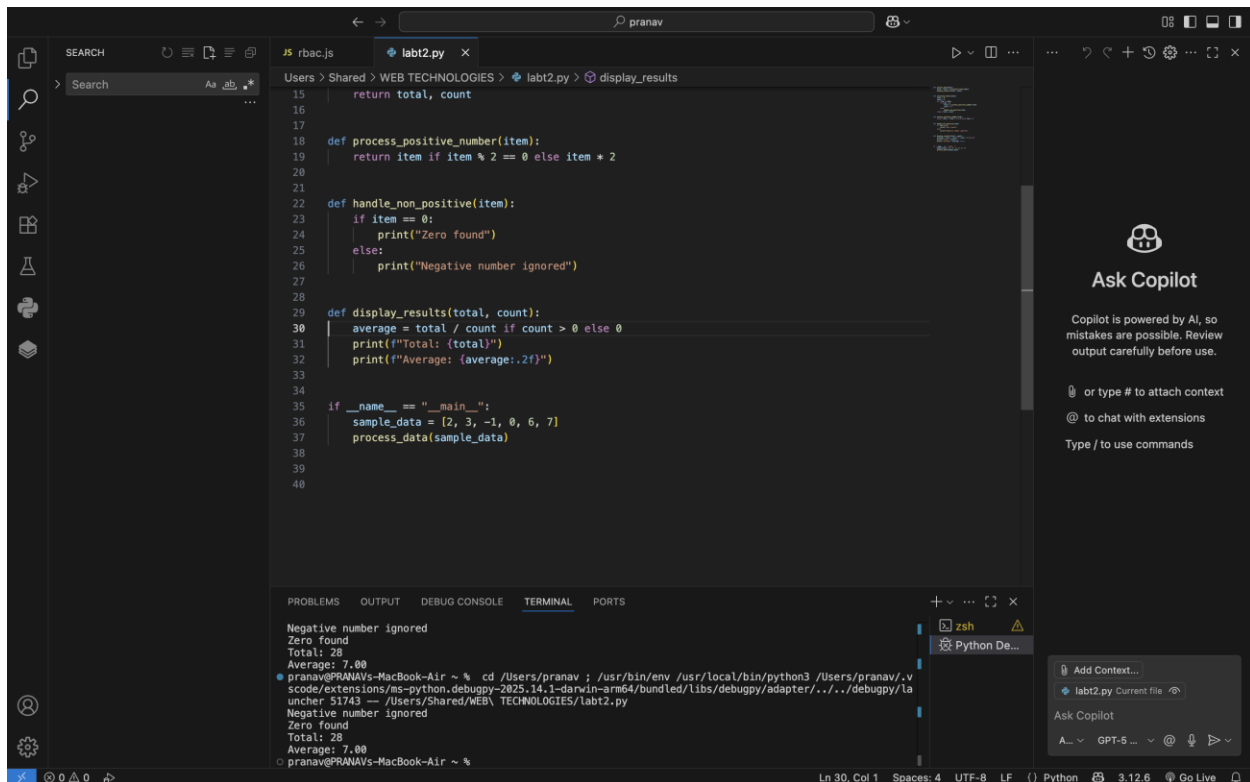
pranav@PRANAVs-MacBook-Air ~ %

FINALIZED CLEAN CODE :

```
1 def process_data(data):
2     total, count = calculate_totals(data)
3     display_results(total, count)
4
5
6 def calculate_totals(data):
7     total = 0
8     count = 0
9     for item in data:
10        if item > 0:
11            total += process_positive_number(item)
12            count += 1
13        else:
14            handle_non_positive(item)
15    return total, count
16
17
18 def process_positive_number(item):
19     return item if item % 2 == 0 else item * 2
20
21
22 def handle_non_positive(item):
23     if item == 0:
24         print("Zero found")
25     else:
26         print("Negative number ignored")
27
28
29 def display_results(total, count):
30     avg = total / count if count > 0 else 0
31     print(f"Total: {total}")
32     print(f"Average: {avg:.2f}")
33
```

Negative number ignored
Zero found
Total: 28
Average: 7.00

pranav@PRANAVs-MacBook-Air ~ %

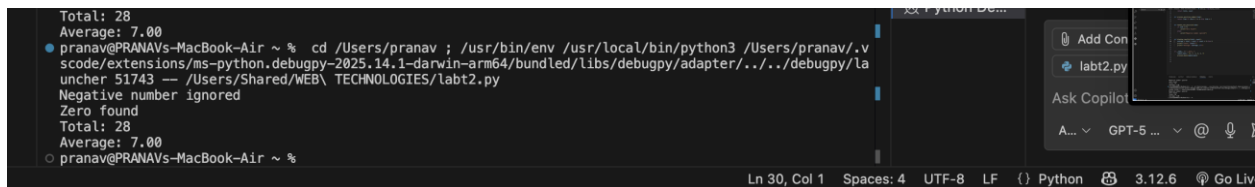


```
15     return total, count
16
17
18 def process_positive_number(item):
19     return item if item % 2 == 0 else item * 2
20
21
22 def handle_non_positive(item):
23     if item == 0:
24         print("Zero found")
25     else:
26         print("Negative number ignored")
27
28
29 def display_results(total, count):
30     average = total / count if count > 0 else 0
31     print(f"Total: {total}")
32     print(f"Average: {average:.2f}")
33
34
35 if __name__ == "__main__":
36     sample_data = [2, 3, -1, 0, 6, 7]
37     process_data(sample_data)
38
39
40
```

Terminal Output:

```
Negative number ignored
Zero found
Total: 28
Average: 7.00
pranav@PRANAVs-MacBook-Air ~ % cd /Users/pranav ; /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/ms-python.debugpy-2025.14.1-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51743 -- /Users/Shared/WEB\ TECHNOLOGIES/labt2.py
Negative number ignored
Zero found
Total: 28
Average: 7.00
pranav@PRANAVs-MacBook-Air ~ %
```

OUTPUT :



```
Total: 28
Average: 7.00
pranav@PRANAVs-MacBook-Air ~ % cd /Users/pranav ; /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/ms-python.debugpy-2025.14.1-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51743 -- /Users/Shared/WEB\ TECHNOLOGIES/labt2.py
Negative number ignored
Zero found
Total: 28
Average: 7.00
pranav@PRANAVs-MacBook-Air ~ %
```

OBSERVATION :

Aspect, Before, After, Explanation

Structure,

Before: Single long, nested function,AFTER :

Split into multiple helper functions,

> Improves readability, allows reuse, and isolates logic.

Naming,

BEFORE : Generic variables and function name, AFTER :

Descriptive names (calculate_totals, process_positive_number),

>Clarifies purpose of each part.

Comments & Docstrings,

BEFORE : None,

AFTER : Added clear docstrings,

>Helps other developers understand purpose of each function.

Complexity, BEFORE :

Deeply nested if-else blocks, AFTER :Moved conditions into smaller functions,

>Reduces cognitive load and makes debugging easier.

Maintainability

BEFORE :Hard to modify or test,

AFTER :

Modular, self-contained functions,

> Easier to update or extend individual parts.